

Confronto di Modelli LLM per l'Espansione di Prompt: Valutazione della Coerenza Semantica tramite CLIP Score

Marco De Stefano

October 2025

1 Introduzione

La prompt expansion è una tecnica di prompt engineering che consiste nel far descrivere a un modello linguistico (LLM) una parola o una frase arricchendola di dettagli, con l'obiettivo di generare risposte più precise e contestualizzate. Questa serve, quindi, a:

1. Dare più contesto → il modello capisce meglio cosa vuoi.
2. Aumentare la precisione → evita risposte troppo vaghe.
3. Controllare il tono e lo stile → puoi farlo scrivere “come un professore”, “come un tecnico”, ecc.
4. Guidare la creatività del modello → utile per generare idee, testi, o codice più rilevante.

La prompt expansion può essere fatta:

- manualmente, da chi scrive il prompt.
- automaticamente, da un altro modello o da uno script, che prende il prompt breve e lo arricchisce prima di mandarlo al LLM.

Nel laboratorio si utilizzerà la seconda tecnica creando uno script che utilizza un certo prompt da noi creato.

Quindi, questo studio ha l'obiettivo di identificare quale modello linguistico sia più efficace per la prompt expansion in contesti di generazione di immagini.

In particolare, vengono confrontati quattro LLM (Mistral, Phi3, Llama3.2 e Gemma2) nella capacità di trasformare nomi semplici di classi (es. "golden retriever") in descrizioni dettagliate e semanticamente coerenti.

La valutazione viene condotta su 1000 classi del dataset ImageNet utilizzando il CLIP Score come metrica principale, che misura la similarità coseno tra gli

embedding visivi delle immagini e gli embedding testuali delle descrizioni generate. I risultati vengono analizzati attraverso distribuzioni statistiche, grafici cumulativi e metriche di performance per determinare quale modello offre il miglior compromesso tra qualità delle espansioni e velocità di elaborazione.

Il CLIP Score è stato scelto come metrica di valutazione poiché misura la coerenza semantica tra testo e immagine nello stesso spazio di rappresentazione utilizzato da molti modelli text-to-image.

Utilizziamo CLIP Score per valutare il potenziale delle espansioni generate, senza effettuare la generazione effettiva di immagini tramite Stable Diffusion. Molti modelli text-to-image (incluso Stable Diffusion in alcune versioni) usano CLIP o architetture simili, di conseguenza se il prompt ha un alto CLIP Score con immagini reali, probabilmente avrà anche successo con la Stable Diffusion (è un programma di intelligenza artificiale che trasforma testo in immagini.)

Questa scelta metodologica consente di:

- Valutare rapidamente migliaia di prompt senza il costo computazionale della generazione immagini
- Mantenere oggettività nella valutazione (metrica automatica vs giudizio umano)
- Confrontare i modelli su un ampio dataset (1000 classi)

1.1 Strumenti

Nel corso dello sviluppo dello script sono state impiegate diverse librerie Python, ognuna con un ruolo specifico nel processo di elaborazione e analisi dei dati. Le principali sono le seguenti:

1.1.1 Librerie

- **matplotlib:** utilizzata per la creazione di grafici e visualizzazioni.
- **openpyxl:** impiegata per la generazione e la gestione di file Excel, in cui sono stati salvati i risultati finali e i valori intermedi delle elaborazioni.
- **transformers:** libreria sviluppata da Hugging Face, utilizzata per caricare e impiegare modelli di deep learning preaddestrati, in particolare per le attività di elaborazione di testo o immagini. In questo caso è stato utilizzato ClipScore, ovvero, metrica basata sul modello CLIP (*Contrastive Language-Image Pre-training*) utilizzata per valutare la coerenza semantica tra descrizioni testuali e immagini. La misura si basa sulla similarità tra le rappresentazioni (embedding) generate dal modello per il testo e per l'immagine, fornendo una stima dell'associazione tra immagine e testo.
- **PIL (Python Image Library):** attraverso il modulo Pillow, consente di aprire, modificare e salvare immagini.

- **pandas:** usata per la lettura del file excel generato.
- **numpy:** impiegata per la gestione di array numerici e per il calcolo matematico efficiente, fondamentale per le operazioni vettoriali e matriciali.
- **torch:** libreria del framework PyTorch, utilizzata per l'implementazione e l'esecuzione di modelli di machine learning e deep learning. Per individuare ed utilizzare risorse computazionali per calcoli più veloci.
- **random:** utilizzata per la generazione di numeri casuali.

1.1.2 Applicativi utilizzati

Oltre alle librerie precedentemente descritte, è stato impiegato un applicativo di notevole importanza chiamato **Ollama**. Esso consente di eseguire in locale modelli di tipo *Large Language Model* (LLM), offrendo la possibilità di effettuare inferenze direttamente sulla macchina senza la necessità di connessioni esterne o risorse cloud. Grazie a questa piattaforma, è stato possibile utilizzare diversi modelli di linguaggio di ultima generazione, ciascuno con specifiche caratteristiche architetturali e prestazionali.

Come menzionato prima si è usato i seguenti modelli:

- **phi3**
- **gemma2**
- **mistral**
- **llama3.2**

1.1.3 Dataset

Nel laboratorio è stato inoltre utilizzato un **dataset derivato da ImageNet**, composto da **1000 classi**, ciascuna contenente **5 immagini**. Le classi rappresentano categorie generiche e diversificate, che spaziano da immagini di animali a oggetti di uso comune.

2 Pipeline

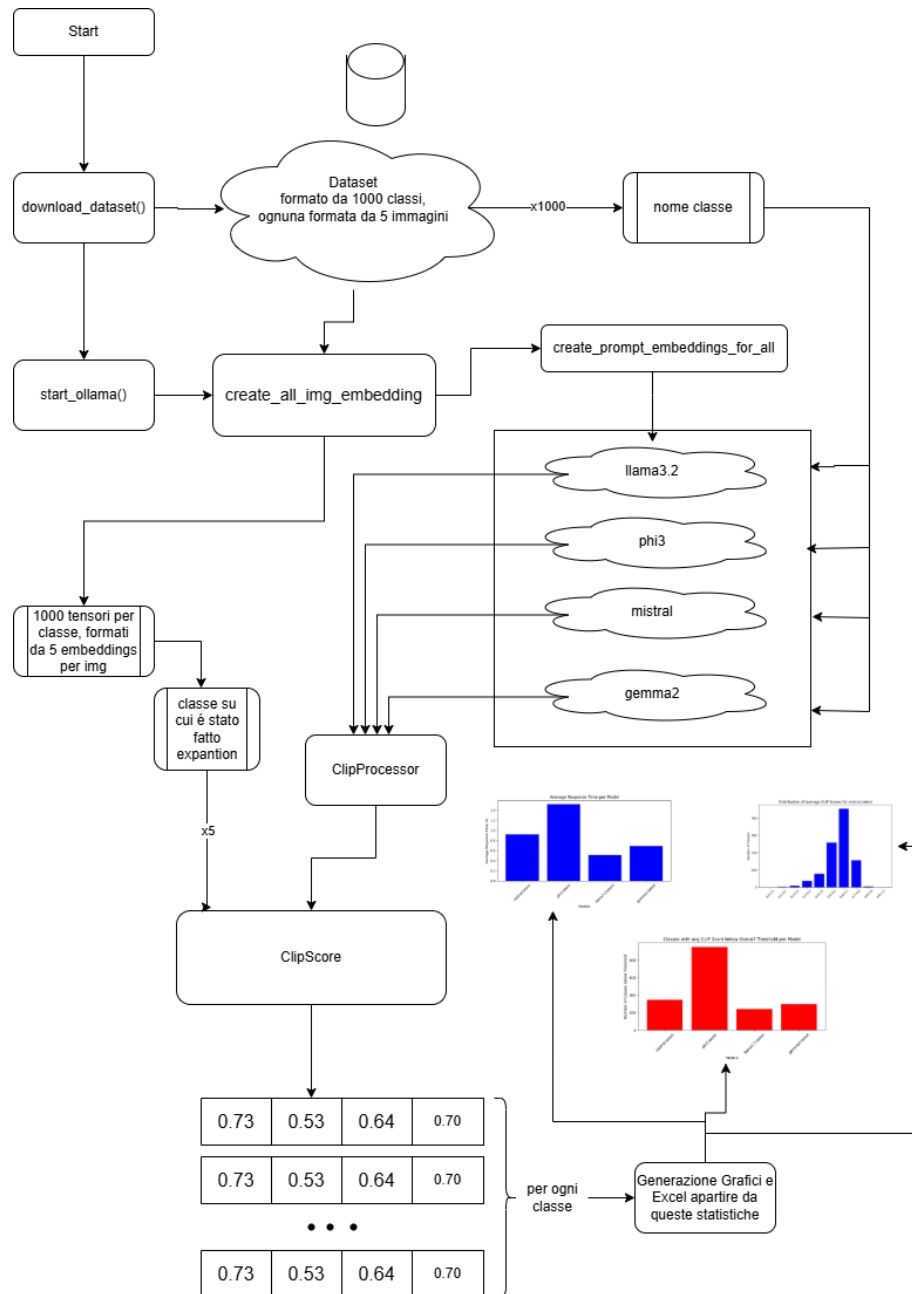


Figura 1: Pipeline del sistema

La pipeline del sistema, illustrata in Figura 1, si articola nei seguenti passaggi:

1. Inizializzazione e Download del Dataset Lo script inizia verificando la presenza del dataset ImageNet localmente. Se non presente, viene eseguito il clone del repository GitHub contenente le 1000 classi con 5 immagini ciascuna. Successivamente viene avviato il server Ollama sulla porta 11434 per gestire le richieste ai modelli LLM.

2. Caricamento del Modello CLIP Viene caricato il modello CLIP (openai/clip-vit-base-patch16) insieme al suo processore. Il modello viene trasferito sul dispositivo disponibile (GPU se presente, altrimenti CPU) per ottimizzare le prestazioni.

3. Generazione degli Embedding delle Immagini Per ogni classe del dataset (1000 classi):

- Vengono caricate le 5 immagini della classe
- Ogni immagine viene processata dal CLIP processor
- Vengono generati gli embedding visivi utilizzando `get_image_features()`
- Gli embedding vengono salvati in file .pt nella directory `./images_embedding/class_i/`

Questo processo viene eseguito una sola volta e gli embedding vengono riutilizzati nelle analisi successive. Quindi, per ogni classe si avrà un file .pt dove si ha gli embedding delle cinque immagini contenute nella cartella di quella classe.

4. Generazione delle Prompt Espanse Per ogni modello LLM (mistral, phi3, llama3.2, gamma2):

- Il modello viene scaricato tramite Ollama se non già presente
- Per ciascuna delle 1000 classi:
 - Viene estratto il nome della classe (es. "golden retriever")
 - Il nome viene inviato al modello con un prompt che richiede di espanderlo in una descrizione dettagliata per Stable Diffusion
 - Vengono generate 3 diverse espansioni per classe per aumentare la robustezza (i risultati mostrati sono con tre diverse espansioni ma può essere fatto anche con una espansione per classe)
 - Ogni espansione viene processata da CLIP per generare l'embedding testuale
 - Gli embedding testuali vengono salvati in una struttura dati
- Viene misurato il tempo medio di risposta per ogni modello

Il prompt utilizzato include esempi di espansioni corrette (**few-shot learning**) per guidare il modello nella generazione. Questa ultima, serve a far capire al LLM cosa serve all'utente senza dover addestrarla.

5. Calcolo dei CLIP Score Per ogni classe e per ogni modello:

- Vengono caricati gli embedding delle 5 immagini della classe
- Vengono recuperati gli embedding delle 3 prompt espanse generate dal modello
- Per ogni coppia immagine-testo viene calcolato il CLIP Score usando la formula:

$$\text{CLIP Score} = w \cdot \max(\text{cosine_similarity}(\text{emb_img}, \text{emb_text}), 0)$$

dove $w = 2.5$ è un fattore di scala. La similarità cosinusoidale rappresenta la similarità angolare tra due vettori, non la loro distanza. I valori di questa operazioni vanno da $[-1, 1]$ e si è deciso di prendere lo zero nel caso di valore negativo poiché non si vuole penalizzare la media.

- Viene calcolata la media dei 15 score ($5 \text{ immagini} \times 3 \text{ prompt}$) per ottenere lo score medio della classe per ogni modello
- Viene calcolata la deviazione standard per ogni modello affinché si possa valutare la consistenza

6. Generazione dei Risultati I risultati vengono elaborati e visualizzati attraverso:

- **Tabella Excel:** contiene per ogni classe e modello lo score medio e la deviazione standard, con formattazione condizionale che evidenzia in verde i valori sopra la media globale e in rosso quelli sotto.
- **Grafici di Distribuzione:** mostrano come si distribuiscono gli score per ogni modello.
- **Distribuzioni Cumulative:** indicano quante classi hanno score inferiori a determinate soglie.
- **Grafico dei Tempi di Risposta:** confronta l'efficienza computazionale dei modelli.
- **Istogramma delle Classi Sotto Soglia:** mostra per ogni modello quante classi hanno performance inferiori alla media globale.

3 Architettura del Codice

Il progetto è strutturato in quattro moduli Python che implementano una pipeline modulare per la valutazione dei modelli LLM:

3.1 Struttura dei Moduli

- **main.py** - Orchestratore principale che coordina l'intera pipeline di esecuzione
- **env_for_chatting.py** - Gestisce la comunicazione con Ollama e la costruzione dei prompt
- **analysis.py** - Calcola gli embedding CLIP e i CLIP Score
- **grafic_analysis.py** - Genera visualizzazioni e tabelle Excel dei risultati

3.2 Modulo: env_for_chatting.py

Questo modulo gestisce l'interazione con i modelli LLM tramite l'API di Ollama.

3.2.1 Componenti Principali

Prompt Engineering: Il sistema utilizza un prompt strutturato (**PREPARED_PROMPT**) che istruisce l'LLM a generare descrizioni realistiche e dettagliate per Stable Diffusion. Include 13 esempi di riferimento (**EXAMPLES**) che implementano few-shot learning, mostrando al modello come espandere correttamente prompt brevi in descrizioni complete.

```
✓ PREPARED_PROMPT = "You are a helpful AI assistant designed to help the user produce prompts that will be used to generate images using Stable Diffusion." \
" Help the user by modifying a given prompt adding many details so that the generated images will look better. " \
"Generate only realistic prompts. Do not engage in artistic and unrealistic prompts."

✓ EXAMPLES = [
    ('a portrait of a blonde woman', 'portrait of a pretty blonde woman, a flower crown, earthy makeup, flowing maxi dress with colorful patterns and fringe, a sunset or nature scene, green and gold color palette, cinematic lighting'),
    ('a portrait of an old man', 'photorealistic, visionary portrait of a dignified older man with weather-worn features, digitally enhanced, high contrast, chiaroscuro lighting technique, intimate, close-up'),
    ('a realistic picture of a living room', 'a living room, bright modern Scandinavian style house, large windows, magazine photoshoot, 8k, studio lighting'),
    ('a closeup of a goth woman', 'closeup portrait photo of beautiful goth woman, makeup, 8k uhd, high quality, dramatic, cinematic'),
    ('a picture of a rabbit in a forest', 'close up photo of a rabbit, forest in spring, haze, halation, bloom, dramatic atmosphere, centred, rule of thirds, 200mm 1.4f macro shot'),
    ('a photo of an indian girl', 'happy indian girl, portrait photography, beautiful, morning sunlight, smooth light, shot on kodak portra 200, film grain, nostalgic mood'),
    ('a professional picture of a luxury bag', 'breathtaking shot of a bag, luxury product style, elegant, sophisticated, high-end, luxurious, professional, highly detailed'),
    ('johnny depp, noir style', 'johnny depp photo portrait, film noir style, monochrome, high contrast, dramatic shadows, 1940s style, mysterious, cinematic'),
    ('a cinematic shot of a cat in the snow', 'a cat under the snow with blue eyes, covered by snow, cinematic style, medium shot, professional photo, animal'),
    ('tokyo, long exposure', 'long exposure photo of tokyo street, blurred motion, streaks of light, surreal, dreamy, ghosting effect, highly detailed'),
    ('a photoshoot of a model, cyberpunk style', 'a glamorous digital magazine photoshoot, a fashionable model wearing avant-garde clothing, set in a futuristic cyberpunk rooftop environment, with a neon cityscape in the background'),
    ('floral tea', 'freshly made hot floral tea in glass kettle on the table, angled shot, midday warm, Nikon D850 105mm, close-up'),
    ('a picture of a smiling girl, red hair, upper body shot', 'masterpiece, best quality, girl, collarbone, wavy hair, looking at viewer, blurry foreground, upper body, necklace, contemporary, plain background')
```

Figura 2: Si utilizza queste due stringhe per consigliare al modello.

Le **funzioni chiave** sono:

- `setup_model(model_name)` - Verifica la presenza del modello su Ollama e lo scarica se necessario.

```
# With this funct we pull some models, choosing the name of LLM
def setup_model(model_name):
    #print(f"Check if '{model_name}' is already present")
    response = requests.get("http://localhost:11434/api/tags")
    installed_models = []
    for m in response.json().get("models", []):
        # We delete the last part of the model name to append the model name without the version
        installed_models.append(m["name"])
    checked = False
    for m in installed_models:
        if m == model_name:
            checked = True
            #print("Model is already present")
            break
    if checked == False:
        print("Model is not already present, we pull it...")
        pull_response = requests.post("http://localhost:11434/api/pull", json={"name": model_name}, stream=True)

        if pull_response.status_code != 200:
            raise RuntimeError(
                f"Failed Pull for the model '{model_name}'. Error Code: '{pull_response.status_code}'")
        else:
            for line in pull_response.iter_lines():
                if line:
                    print(line.decode('utf-8'))

        print(f"Model '{model_name}' is successfully installed.")
```

Figura 3: Codice della funzione `setup_model`

- `chat_with_model(class_name, model_name, message_prompt)` - Invia richieste HTTP POST all'API di Ollama e gestisce le risposte.

```
# Function used to chat with the model
# It sends a request to the model with the image and the message prompt
def chat_with_model(class_image, model_name, message_prompt):
    # setup_model(model_name)
    input_string = message_prompt + class_image
    input_string = input_string.replace("\n", " ")

    try:
        start = time.time()
        response = requests.post("http://localhost:11434/api/generate",
                                json={"model": model_name, "prompt": input_string, "stream": False})
        response_from_chat = response.json()
        end = time.time()
        t = end - start
        # Verifica chiave 'response'
        if "response" in response_from_chat:
            return response_from_chat["response"], t
        elif "error" in response_from_chat:
            print(f"Error from model {model_name}: {response_from_chat['error']}")
            return "", t
        else:
            raise KeyError(f"Unexpected response: {response_from_chat}")
    except requests.exceptions.ConnectionError:
        pass
```

Figura 4: Codice della funzione `chat_with_model`

- `choose_class_and_img(class_idx, img_idx)` - Seleziona un'immagine specifica dal dataset e recupera il nome della classe

3.3 Modulo: analysis.py

Implementa il calcolo degli embedding CLIP e la metrica di coerenza semantica. Ma prima di procedere, è utile comprendere cos'è CLIP e come funziona.

CLIP (Contrastive Language-Image Pre-training) è un modello di deep learning sviluppato da OpenAI che collega immagini e testo in uno spazio semantico condiviso, essendo stato addestrato su milioni di coppie immagine-testo.

Il flusso di elaborazione prevede due fasi principali: **preprocessing** ed **embedding**.

Nella fase di **preprocessing**, la funzione `clip_processor` prepara sia l'immagine che il testo per l'elaborazione da parte del modello. Per il testo, questo processo include la tokenizzazione (ovvero l'associazione di un identificativo numerico a ogni parola), l'aggiunta di token speciali, e infine il padding per uniformare la lunghezza delle sequenze. Per l'immagine, il preprocessing comporta il ridimensionamento a dimensioni fisse, la normalizzazione dei valori dei pixel nei canali RGB secondo media e deviazione standard specifiche, e la conversione in tensori.

Successivamente, nella fase di **embedding**, vengono utilizzate le funzioni `clip_model.get_text_features()` e `clip_model.get_image_features()` che generano rappresentazioni vettoriali (embeddings) rispettivamente del testo e dell'immagine. Queste funzioni passano i dati preprocessati attraverso reti neurali profonde. Gli embeddings risultanti, tipicamente vettori di 512 dimensioni, si trovano nello stesso spazio e possono quindi essere confrontati direttamente per misurare la similarità tra contenuto testuale e visivo.

3.3.1 Pipeline di Embedding

1. Embedding delle Immagini:

- Funzione: `create_all_img_embedding()`
- Processa 5000 immagini (1000 classi $\times$ 5 immagini)
- Genera embedding di dimensione 512 tramite CLIP ViT-base-patch16
- Salva i risultati in `./images_embedding/class.i/embeddings.pt`
- Eseguito una sola volta (caching per successive esecuzioni)

```

#Create a file than calculates embeddings of images for all classes.
def create_all_img_embedding(clip_processor, clip_model, device, num_classes, num_img_4_class, name_model):
    tokenizer = CLIPTokenizer.from_pretrained(name_model)
    tokenizer.use_fast = False

    if os.path.exists('./images_embedding') == False:
        os.mkdir('./images_embedding')
        for i in tqdm(range(num_classes), desc="Calculating embeddings of all images"):
            class_embeddings = []
            for j in range(num_img_4_class):
                img = efc.choose_class_and_img(i, j)
                image_path = img['path']
                image = Image.open(image_path).convert("RGB")
                image_inputs = clip_processor(images=image, return_tensors="pt")
                image_inputs = {k: v.to(device) for k, v in image_inputs.items()}
                with torch.no_grad():
                    img_emb = clip_model.get_image_features(**image_inputs)
                #I use squeeze to reduce the dimensionality of the tensor, then i save tensor copy in the CPU.
                #compatibility with pcs without a GPU
                img_emb = img_emb.squeeze(0).cpu()
                class_embeddings.append(img_emb)

            # Stack the 5 embeddings in one tensor [5, D]
            class_embeddings = torch.stack(class_embeddings)

            # Save the tensor in a file .pt in a directory for the class
            class_dir = f'./images_embedding/class_{i}'
            if not efc.os.path.exists(class_dir):
                efc.os.mkdir(class_dir)

            save_path = f'{class_dir}/embeddings.pt'
            torch.save(class_embeddings, save_path)

            #print(f'Saved embeddings for class {i} at {save_path}')
    else:
        print('The Embedding Directory is already create, also the embedding of all image of Dataset.')

```

Figura 5: Codice della funzione create_all_img_embedding

2. Embedding dei Prompt:

- Funzione: create_prompt_embedding_for_all()
- Per ogni modello LLM, genera 3(dipende dal parametro messo) espansioni per ciascuna delle 1000 classi
- Totale: 12000 query (4 modelli $\times$ 1000 classi $\times$ 3 prompt)
- Converte i testi in embedding CLIP di dimensione 512
- Gestisce il limite di 77 token tramite `truncation=True`
- Misura il tempo medio di risposta per modello

```

# Function to calculate embeddings of a single class -> put the prompt in the models.
# So if the class number is 1000 and we have 5 models we have to do 5000 prompt embeddings.

def create_prompt_embedding_for_all(clip_processor, clip_model, model_name, models, class_num, device, num_prompts=3):
    struct_of_text_embeddings = {}
    for _ in range(len(models)):
        for _ in range(class_num) # class_num * num_models

    tokenizer = CLIPTokenizer.from_pretrained(model_name)
    tokenizer.use_fast = True
    message_prompt = efc.build_message_prompt()
    # To measure the average time to compute the embedding of a prompt for each model
    avg_time = [0.0]*len(models)

    for i in range(len(models)):
        efc.setup_model(models[i])
        time_start = time.time()
        for j in tqdm(range(class_num), desc=f"Calculating prompt extension and text embeddings for model {models[i]}"):
            img, efc.choose_class_and_img(j, i)
            captions = []
            # This is the prompt extension of the class
            print(img['class name'])
            for _ in range(num_prompts): # different captions for each class
                caption, t=efc.chat_with_model(img['class name'], models[i], message_prompt)
                if models[i] == "deepseek-r1:14b":
                    caption = efc.extract_short_long(caption)
                captions.append(caption)
            text_inputs = clip_processor(text=captions, return_tensors="pt", padding=True, truncation=True, max_length=77)
            text_inputs = {k: v.to(device) for k, v in text_inputs.items()}
            with torch.no_grad():
                text_embs = clip_model.get_text_features(**text_inputs)
            for emb in text_embs:
                struct_of_text_embeddings[j][i].append(emb.cpu())
            print(f"Saved embedding for class {j} and model {models[i]}")
        time_end = time.time()
        avg_time[i] = (time_end - time_start) / (class_num*num_prompts)
    return struct_of_text_embeddings, avg_time

```

Figura 6: Codice della funzione create_prompt_embedding_for_all()

3.3.2 Calcolo del CLIP Score

La funzione `clip_score(emb_img, emb_text, w=2.5)`, come é stato detto prima, implementa la metrica:

$$\text{CLIP Score} = w \cdot \max(\cos(\mathbf{v}_{\text{img}}, \mathbf{v}_{\text{text}}), 0) \quad (1)$$

dove:

- $\mathbf{v}_{\text{img}}$ e $\mathbf{v}_{\text{text}}$ sono embedding normalizzati
- $\cos(\cdot, \cdot)$ è la similarità coseno (misura l'angolo tra i vettori)
- $w = 2.5$ è un fattore di amplificazione empirico
- $\max(\cdot, 0)$ elimina correlazioni negative

Per ogni classe, viene calcolata la media su 15 coppie (5 immagini $\times$ 3 prompt) e la deviazione standard per misurare la consistenza, come vediamo dal codice sotto.

```

# Embedding text is one vector of one class
# Models is the list of models
# Embeddings image is the tensor of all embeddings of images of one class [5,0]
def calculate_clip_score(embedding_text, models, embeddings_image):
    average_clip_scores = [
        {
            "model": j,
            "scores": [],
            "avg_score": 0.0, # la media
            "deviation": 0.0 # la deviazione standard
        }
        for j in range(len(models))]

    for emb_img in embeddings_image:
        for i in range(len(models)):
            score = 0.0
            for k in range(len(embedding_text[i])):
                score += clip_score(emb_img, embedding_text[i][k])
            score /= len(embedding_text[i])
            average_clip_scores[i]["scores"].append(score)

    # Calculate average scores
    for entry in average_clip_scores:
        if entry["scores"]:
            entry["avg_score"] = sum(entry["scores"]) / len(entry["scores"])
            entry["deviation"] = np.std(entry["scores"])
        else:
            entry["avg_score"] = 0.0

    return average_clip_scores

```

Figura 7: Media dei Clipscore

3.4 Modulo: `grafic_analysis.py`

Genera output visivi e tabulari per l'analisi comparativa.

3.4.1 Output Prodotti

- **ClipScores.xlsx** - Tabella Excel con medie e deviazioni standard per ogni classe e modello, con formattazione condizionale (verde per score sopra la media globale, rosso sotto)
- **Distribuzioni** - Istogrammi che mostrano come si distribuiscono gli score in 10 bin (0.0-0.1, 0.1-0.2, ..., 0.9-1.0)
- **Distribuzioni Cumulative** - Grafici che mostrano quante classi hanno score minore uguale della soglia per ogni valore di soglia.
- **Tempi di Risposta** - Istogramma dei tempi medi di elaborazione per modello
- **Classi Sotto Soglia** - Conteggio delle classi con performance inferiori alla media globale

3.5 Modulo: `main.py`

Coordina l'esecuzione sequenziale della pipeline.

3.5.1 Flusso di Esecuzione

1. Inizializzazione

- Download del dataset ImageNet-1000 (se non presente)
- Avvio del server Ollama sulla porta 11434

- Rilevamento GPU/CPU

2. Caricamento Modelli

- Caricamento di CLIP (openai/clip-vit-base-patch16)
- Setup dei 4 modelli LLM tramite Ollama

3. Generazione Embedding

- Calcolo embedding immagini (con caching)
- Generazione prompt espanse e relativi embedding

4. Analisi

- Calcolo CLIP Score per tutte le combinazioni classe-modello
- Aggregazione statistiche (media, deviazione standard)

5. Visualizzazione

- Generazione tabelle Excel
- Creazione grafici comparativi

3.6 Considerazioni Implementative

- **Caching:** Gli embedding delle immagini vengono salvati su disco per evitare ricalcoli
- **Accelerazione GPU:** Quando disponibile, CUDA viene utilizzato per il calcolo degli embedding
- **Ottimizzazione Caricamento Modelli:** Il sistema elabora tutte le 1000 classi con un modello prima di passare al successivo, evitando ripetuti caricamenti/scaricamenti da Ollama. Questo approccio riduce le operazioni di switching da 4000 a 4, migliorando significativamente le performance temporali.

4 Risultati

4.1 Distribuzione dei CLIP Score

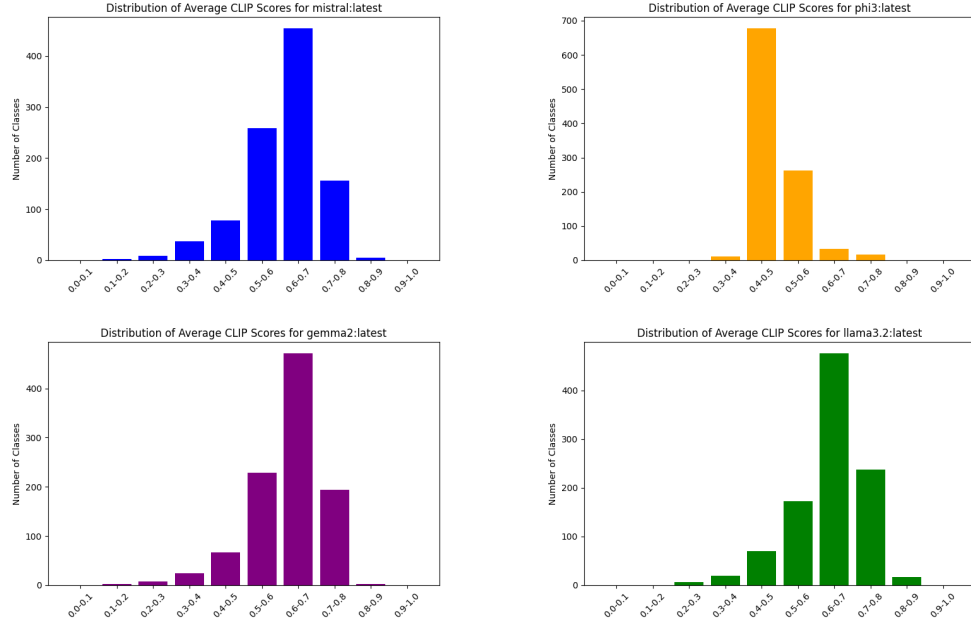


Figura 8: Distribuzione dei CLIP Score per tutti i nostri modelli test

Dalle distribuzioni emergono pattern distintivi per ciascun modello:

- **Mistral:** mostra una distribuzione concentrata nella fascia 0.6-0.7, con circa 450 classi in questo range, indicando prestazioni consistenti ma non eccellenti.
- **Phi3:** presenta la distribuzione più concentrata, con circa 680 classi nella fascia 0.4-0.5, suggerendo espansioni meno dettagliate o meno allineate semanticamente con le immagini.
- **Gemma2:** ha una distribuzione più bilanciata, con il picco a 0.6-0.7 (circa 480 classi) ma anche una buona presenza nelle fasce superiori.
- **Llama3.2:** mostra la distribuzione più promettente, con il picco principale a 0.6-0.7 (circa 480 classi) e una presenza significativa nella fascia 0.7-0.8 (circa 240 classi).

4.2 Distribuzioni Cumulative

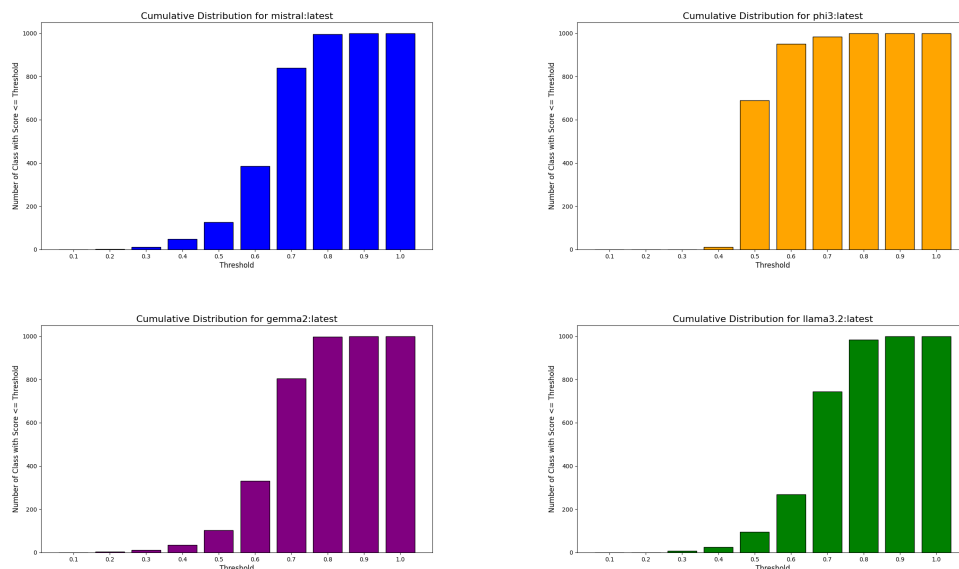


Figura 9: Distribuzione Cumulativa dei CLIP Score per tutti i modelli

In questi grafici, disegniamo le distribuzioni cumulative dove sull'asse y ci sono il numero di classi che sono minori o uguali del threshold mentre sull'asse x ci stanno i threshold possibili di score. Notiamo, quindi, che:

- **Mistral:** ha delle classe che stanno anche sotto lo 0.2, quindi, può capitare che ci siano allucinazioni.
- **Phi3:** sembra avere sicuramente una robustezza migliore alle allucinazioni, non dando però un contributo significativo nella descrizione delle foto poiché dal grafico 8 vediamo che i clipscore oscillano tra lo 0.4 e lo 0.5.
- **Gemma2:** ha una distribuzione cumulativa simile a quella del modello **Mistral**.
- **Llama3.2** simile a Mistral e Gemma2.

Nota importante: Phi3 mostra maggiore robustezza alle allucinazioni (poche classi sotto 0.3) ma genera descrizioni troppo generiche (concentrate 0.4-0.5). Per applicazioni di generazione immagini, score nell'intervallo 0.6-0.7 sono preferibili a 0.4-0.5, anche a costo di maggiore variabilità.

4.3 Tempi di Risposta

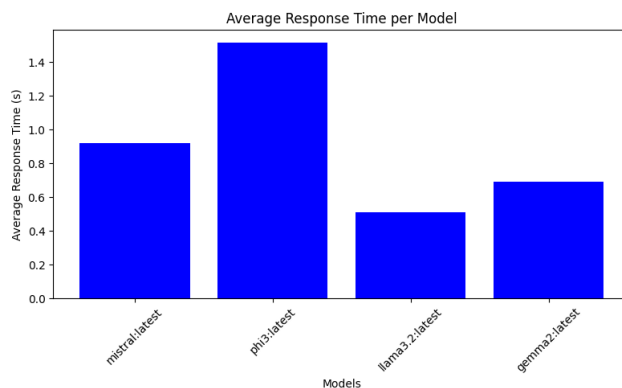


Figura 10: Tempi medi di risposta per modello

In questo grafico si descrive, invece, al variare del modello sull'asse x i tempi medi di risposta per una query. Quindi, dai tempi di risposta si capisce che:

- **Llama3.2** è il più veloce (circa 0.5s per prompt)
- **Gemma2** e **Mistral** hanno tempi intermedi (0.7-0.9s)
- **Phi3** è il più lento (circa 1.5s per prompt)

4.4 Classi Sotto Soglia

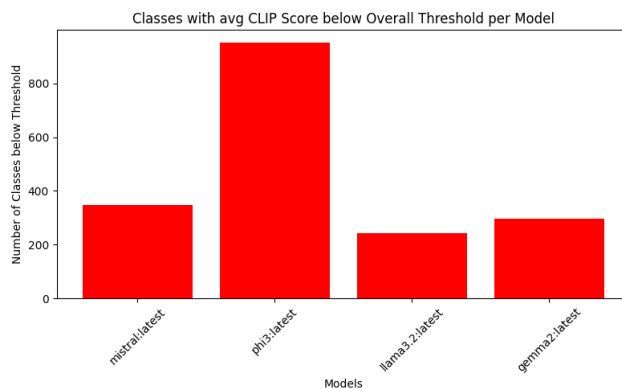


Figura 11: Numero di classi con CLIP Score sotto la soglia media globale

Come penultimo grafico, guarderemo un risultato che generalizza quello che abbiamo detto fino ad ora se calcoliamo la media globale del clip score. Infatti vediamo che l'istogramma mostra un andamento di questo genere:

- **Phi3** ha il maggior numero di classi sotto soglia (circa 950)
- **Mistral** e **Gemma2** hanno circa 340 e 290 classi sotto soglia rispettivamente
- **Llama3.2** ha il minor numero di classi sotto soglia (circa 240)

4.5 Media del ClipScore

La seguente analisi rappresenta un esperimento che ho voluto fare per conto mio, realizzando alla fine che fosse fuori scopo. Per completezza ho intenzione di pubblicare i risultati riguardo una classe specifica del nostro dataset.

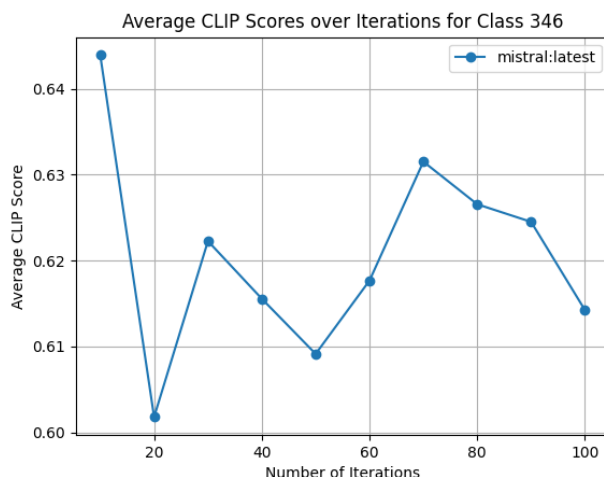


Figura 12: Valore Medio del ClipScore al variare del numero di espansioni (Mistral)

L'obiettivo era quello di capire dopo quanti tentativi di espansione il ClipScore poteva stabilizzarsi. Cosa che é totalmente inutile visto che si tratta di una risposta di un modello su cui non abbiamo un controllo effettivo.

5 Conclusioni

I risultati dimostrano che la scelta del modello per la prompt expansion ha un impatto significativo sulla qualità delle descrizioni generate e, conseguentemente, sulla potenziale qualità delle immagini che potrebbero essere generate da un sistema text-to-image. Dall'analisi complessiva emerge che:

- **Llama3.2** risulta il modello più bilanciato, offrendo il miglior compromesso tra qualità delle espansioni (CLIP Score elevati) e velocità di elaborazione. È il modello raccomandato per applicazioni di prompt expansion.
- **Mistral** offre buone prestazioni in termini di qualità, con tempi di risposta accettabili, risultando una valida alternativa.
- **Gemma2** presenta prestazioni intermedie sia in termini di qualità che di velocità.
- **Phi3**, nonostante sia un modello compatto, mostra le prestazioni più deboli in questo specifico task di prompt expansion, sia in termini di CLIP Score che di tempo di elaborazione.

5.1 Limitazioni e Considerazioni Tecniche

Il modello CLIP utilizzato (openai/clip-vit-base-patch16) presenta una limitazione architetturale significativa: può processare al massimo **77 token** per ogni prompt testuale. Infatti, quando un utente passa un prompt alla rete, un tokenizer scompone la frase in parole ed ad ognuna di essa gli associa un token, dopo di che questi token vengono processati tramite una Deep Neural Network, come detto prima, restituendo un vettore embedding.

Vengono utilizzati solo 77 token e quindi, nel nostro caso, il resto viene troncato, influenzando il risultato del embedding e, quindi, del ClipScore nel caso l'output sia troppo lungo.

Possibili Miglioramenti: Nel codice attuale non abbiamo controllo diretto sulla lunghezza dell'output dei modelli LLM. Tuttavia, potrebbero essere implementate le seguenti strategie:

- **Controllo della temperatura:** Modificare il parametro di temperatura (che controlla il "livello di creatività" del modello) per ottenere output più concisi (Temperatura più bassa).
- **Prompt engineering:** Specificare esplicitamente nel prompt un limite di lunghezza (es. "generate a detailed description in maximum 50 words").
- **Post-processing:** Implementare un sistema di selezione delle frasi più rilevanti prima di passare il testo a CLIP, lo si può fare tramite librerie NLP (come spacy).

- **Uso di modelli CLIP alternativi:** Esistono varianti di CLIP con limiti di token superiori che potrebbero essere esplorate in lavori futuri
- **Possibile Parallelizzazione:** di prompt expansion fatto dal modello.